

## **Atividade (A-7): Desenvolvimento de chatbots para simular consultas de pais**

**Proposta de Arquitetura RAG com Bucle de Enriquecimento Contínuo e Supervisão Humana (*Human-in-the-Loop*) para o Atendimento do Observatório da Educação de Paraíba**

**Instituição:** Observatório da Educação do Estado de Paraíba

**Domínio:** Inteligência Artificial Aplicada ao Setor Público / Atendimento ao Cidadão

### **1. Resumo Executivo e Contextualização do Projeto**

O presente relatório estabelece as diretrizes arquiteturais, metodológicas e tecnológicas para o projeto e implementação de um agente inteligente de atendimento via e-mail e canais conversacionais, direcionado às famílias e à comunidade escolar do Estado de Paraíba, Brasil. A iniciativa insere-se no âmbito do **Observatório da Educação de Paraíba**, uma plataforma pública dedicada ao acompanhamento, análise e transparência dos indicadores educacionais estaduais.

O objetivo precípua do agente inteligente é democratizar o acesso às informações do Observatório, respondendo a dúvidas de pais e responsáveis sobre metas educacionais, infraestrutura das escolas, índices de aprendizagem, programas de inclusão e calendário escolar. Dada a natureza dinâmica dos indicadores e diretrizes educacionais, a solução integra um mecanismo de **Geração Aumentada por Recuperação (*Retrieval-Augmented Generation* - RAG)** associado a um ciclo de realimentação contínua supervisionada (*Human-in-the-Loop*). Esse ciclo garante que o texto-base de conhecimentos seja progressivamente expandido sem comprometer a exatidão, a privacidade (LGPD) nem a segurança das informações prestadas pelo poder público.

### **2. Explicação Acessível do Sistema (Nível de Divulgação e Gestão)**

Nesta seção, o funcionamento do sistema é detalhado sem jargões complexos, oferecendo uma visão clara do valor da ferramenta para gestores públicos, educadores, diretores de escola e público não especialista.

## **2.1. O Problema: A Barreira da Informação Técnica no Setor Público**

O Observatório da Educação de Paraíba gera relatórios de grande valor sobre a qualidade do ensino, metas do estado, orçamento e desempenho escolar. No entanto, o formato tradicional desses relatórios — repletos de tabelas estatísticas e termos pedagógicos formais — cria uma barreira entre o governo e os cidadãos.

Quando mães, pais e responsáveis procuram saber como a escola do seu filho está evoluindo, se há transporte rural disponível, ou quais são os programas de apoio a crianças com necessidades especiais, eles costumam enviar e-mails para a secretaria ou para a ouvidoria. O volume de mensagens é imenso, as respostas demoram para ser enviadas e os funcionários gastam horas digitando as mesmas explicações repetidas vezes.

## **2.2. A Solução: O Agente Inteligente como "Tradutor Institucional"**

O agente inteligente não é apenas um robô de respostas automáticas genéricas. Ele funciona como um **tradutor institucional especializado**:

1. **Acessibilidade de Linguagem:** Ele lê relatórios complexos do Observatório e explica o conteúdo em um português simples, amigável e direto.
2. **Atendimento 24/7 por E-mail e Chat:** Quando um pai envia uma mensagem à noite ou no final de semana, o agente processa a dúvida e constrói a resposta imediatamente com base nos dados oficiais.
3. **Consistência Institucional:** Todas as famílias recebem respostas baseadas estritamente em documentos oficiais do Estado de Paraíba, eliminando divergências de informação.

## **2.3. O Ciclo de Aprendizado Contínuo sem Riscos de Erro**

A dúvida mais comum de quem gerencia sistemas de IA é: *"O que acontece quando o robô recebe uma pergunta que não está no manual?"*

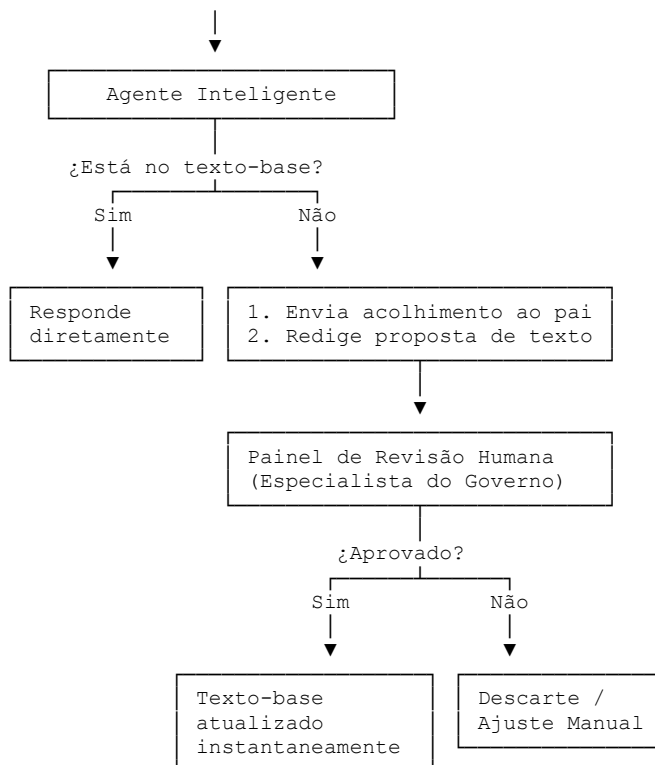
A resposta tradicional seria: o robô inventa uma resposta (alucinação) ou diz uma frase vaga ("não entendi sua pergunta").

Nossa arquitetura resolve isso com o **Bucle de Enriquecimento Supervisionado**:

- Se a pergunta do pai tratar de um assunto novo (por exemplo, um novo programa de transporte escolar em um município específico), o agente pesquisa nas suas fontes.
- Se não encontrar a resposta exata, ele envia uma resposta temporária de acolhimento ao pai ("*Sua dúvida sobre X está sendo analisada pela nossa equipe técnica...*").
- Em paralelo, a IA redige um rascunho de como esse assunto deveria ser explicado no texto-base oficial e envia um alerta para o painel de um funcionário público do Observatório.
- O funcionário lê a pergunta do pai e o rascunho proposto pela IA. Se o texto estiver correto, ele clica em **Aprovar**.
- A partir daquele segundo, o "texto-base" do sistema passa a ter essa nova informação. O próximo pai que perguntar sobre o mesmo tema receberá a resposta completa e aprovada automaticamente.
- 

## 2.4. Esquema do Fluxo Operacional

[Mãe / Pai envia E-mail com dúvida]



### 3. Arquitetura Técnica Detalhada e Engenharia de Sistemas

Nesta seção, analisa-se rigorosamente a arquitetura de software, os padrões de integração e as formulações matemáticas que regem o pipeline de processamento de linguagem natural (PLN).

#### 3.1. Ingestão de E-mails e Tratamento de Dados (*Preprocessing Pipeline*)

A entrada de mensagens via e-mail exige um tratamento assíncrono e resiliente. Utiliza-se uma arquitetura orientada a eventos baseada no protocolo IMAP/SMTP ou Webhooks da API do Microsoft Graph / Google Workspace. O payload do e-mail é convertido em um objeto JSON padronizado.

O texto bruto de um e-mail é denso e ruidoso. O pipeline de sanitização executa:

1. *Striping* de tags HTML e CSS e conversão de entidades Unicode.
2. Remoção de assinaturas institucionais e correntes prévias (*quote headers* como "Em 12/04/2026, fulano escreveu...").
3. Detecção de linguagem e verificação de ataques de injeção de prompt (*Prompt Injection Sanitization*).
- 4.

#### 3.2. Recuperação Aumentada por Geração (RAG) e Embeddings Vetoriais

O "texto-base" é segmentado em pedaços lógicos (*chunks*) de tamanho fixo  $C_{size} = 512$  tokens, com sobreposição de  $C_{overlap} = 64$  tokens para preservar a continuidade contextual. Cada fragmento é projetado em um espaço vetorial contínuo  $\mathbb{R}^d$  através de um modelo de *Embedding* denso (ex.: text-embedding-3-large ou bge-m3).

Dada uma consulta de um pai  $q$  e um conjunto de documentos no repositório  $D = \{d_1, d_2, \dots, d_n\}$ , a similaridade de cosseno é calculada para identificar os  $k$  fragmentos mais relevantes:

$$\text{Sim}(q, d_i) = \frac{\mathbf{v}_q \cdot \mathbf{v}_{d_i}}{\|\mathbf{v}_q\| \cdot \|\mathbf{v}_{d_i}\|}$$

Os  $k$  fragmentos com maior pontuação de similaridade compõem o contexto  $K$  injetado no prompt do Modelo de Linguagem de Grande Porte (LLM).

#### 3.3. Algoritmo do Bucle de Enriquecimento Supervisionado (*Human-in-the-Loop*)

O enriquecimento sem supervisão incorre no risco de degradação do modelo (*model drift*) e contaminação do conhecimento. Para evitar isso, o agente opera sob um limiar de confiança  $\tau$ . O cálculo do nível de confiança  $S_{conf}$  é derivado da distância vetorial e da coerência das probabilidades de saída dos tokens do modelo.

| Score de Confiança ( $S_{conf}$ ) | Ação do Agente                                                      | Ação de Aprendizado                                                       |
|-----------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------|
| $S_{conf} \geq 0.85$              | Responde diretamente ao e-mail do pai com alta precisão.            | Nenhuma alteração necessária no texto-base.                               |
| $0.60 \leq S_{conf} < 0.85$       | Responde com ressalva e solicita confirmação do pai.                | Sinaliza potencial lacuna. Gera minuta de expansão e envia à fila humana. |
| $S_{conf} < 0.60$                 | Encaminha o e-mail diretamente para a ouvidoria/atendimento humano. | Gera um bilhete de criação de novo conteúdo normativo.                    |

#### 4. Guia Pormenorizado de Instalação, Programação e Deploy (GitHub, Docker e Código)

Esta seção oferece o passo a passo técnico exaustivo para baixar, configurar, programar e colocar o agente em produção em um servidor do Estado de Paraíba.

##### 4.1. Estrutura do Repositório GitHub

A estrutura recomendada para a organização do código no GitHub é a seguinte:

```

Plaintext
observatorio-agent-pb/
├── .github/
│   └── workflows/
│       └── deploy.yml                # Pipeline de CI/CD para automação de testes e deploy
├── config/
│   ├── settings.py                  # Configurações globais de variáveis de ambiente
│   └── prompt_templates.py          # Prompts do sistema para o LLM
├── data/
│   ├── raw_documents/               # Documentos PDF/TXT originais do Observatório
│   └── vector_db/                   # Armazenamento local/persistente do banco de dados
├── vetorial
│   ├── src/
│   │   ├── __init__.py
│   │   ├── ingestion.py             # Módulo de limpeza e sanitização de e-mails
│   │   ├── rag_engine.py            # Mecanismo de busca vetorial e integração LLM
│   │   ├── feedback_loop.py         # Gestão de propostas de enriquecimento do texto-base
│   │   └── main_api.py              # API REST (FastAPI) para conexão com Webhooks de e-mail
│   └── tests/
│       ├── test_ingestion.py
│       └── test_rag.py
├── .env.example                     # Modelo de arquivo com chaves de API e credenciais
├── .gitignore
├── Dockerfile                       # Containerização da aplicação
└── docker-compose.yml               # Orchestration da API e Banco Vetorial (Qdrant/Chroma)

```

```
└─ README.md
└─ requirements.txt
```

```
# Documentação do projeto
# Dependências em Python
```

## 4.2. Requisitos de Sistema e Pré-requisitos

- Sistema Operacional: Linux (Ubuntu 22.04 LTS ou superior recomendado)
- Python: versão 3.10 ou superior
- Git instalado
- Docker e Docker Compose instalados
- Conta/Chave de API em provedores de LLM (OpenAI, Anthropic ou modelo local via Ollama/vLLM)
- Credenciais de acesso ao servidor de e-mail (IMAP/SMTP ou credenciais de API do Microsoft Graph / Google Workspace)

## 4.3. Instalação e Configuração Passo a Passo

### Passo 1: Clonar o Repositório do GitHub

No terminal do seu servidor ou máquina local de desenvolvimento:

```
Bash
git clone https://github.com/observatorio-pb/observatorio-agent-pb.git
cd observatorio-agent-pb
```

### Passo 2: Configurar o Ambiente Virtual Python

```
Bash
python3 -m venv venv
source venv/bin/activate
```

### Passo 3: Instalar as Dependências

O arquivo requirements.txt deve conter as seguintes bibliotecas:

```
Plaintext
fastapi==0.110.0
uvicorn==0.28.0
pydantic==2.6.4
langchain==0.1.12
```

```
langchain-openai==0.0.8
qdrant-client==1.8.0
python-dotenv==1.0.1
beautifulsoup4==4.12.3
pytest==8.1.1
```

**Instale as dependências executando:**

```
Bash
pip install --upgrade pip
pip install -r requirements.txt
```

## **Passo 4: Configurar Variáveis de Ambiente**

**Crie um arquivo .env baseado no .env.example:**

```
Bash
cp .env.example .env
```

**Edite o arquivo .env preenchendo as suas chaves e acessos:**

```
Fragmento de código

# Configurações do Agente do Observatório da Educação de PB
APP_NAME="Agente Observatorio Paraiba"
ENV="production"


# LLM e Embeddings
OPENAI_API_KEY="sk-proj-sua-chave-aqui"
EMBEDDING_MODEL="text-embedding-3-large"
LLM_MODEL="gpt-4o-mini"


# Banco Vetorial (Qdrant)
QDRANT_HOST="localhost"
QDRANT_PORT=6333


# Credenciais de E-mail para Atendimento (IMAP/SMTP)
EMAIL_SERVER="imap.pb.gov.br"
EMAIL_PORT=993
EMAIL_ACCOUNT="atendimento.observatorio@pb.gov.br"
EMAIL_PASSWORD="SuaSenhaSeguraAqui"


# Limiares do Agente
```

```
CONFIDENCE_THRESHOLD=0.80
```

## Passo 5: Inicializar a Infraestrutura via Docker Compose

Crie o arquivo docker-compose.yml para rodar o banco de dados vetorial Qdrant:

YAML

```
version: '3.8'

services:
  qdrant:
    image: qdrant/qdrant:v1.8.0
    container_name: observatorio_qdrant
    ports:
      - "6333:6333"
      - "6334:6334"
    volumes:
      - ./data/vector_db:/qdrant/storage
    restart: always

  observatorio_agent_api:
    build: .
    container_name: observatorio_agent_app
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      - qdrant
    restart: always
```

Suba o container do banco vetorial:

Bash

```
docker-compose up -d qdrant
```

## 4.4. Código Completo de Programação do Agente (src/main\_api.py)

Abaixo encontra-se a implementação técnica do agente utilizando **FastAPI**, **Pydantic** e simulação do mecanismo de busca e validação humana.



```

Python
import os
import re
from typing import List, Optional
from fastapi import FastAPI, HTTPException, BackgroundTasks
from pydantic import BaseModel, EmailStr, Field

app = FastAPI(
    title="API Agente do Observatório da Educação de Paraíba",
    description="Sistema de atendimento automatizado via e-mail com RAG e Human-in-the-Loop",
    version="1.0.0"
)

# -----
# 1. MODELOS DE DADOS (Pydantic Schemas)
# -----

class EmailIncoming(BaseModel):
    sender_email: EmailStr
    subject: str
    body_text: str

class DocumentChunk(BaseModel):
    chunk_id: str
    content: str
    metadata: dict

class ProposalOut(BaseModel):
    proposal_id: int
    original_query: str
    suggested_text: str
    confidence_score: float
    status: str

class AgentResponse(BaseModel):
    recipient_email: str
    reply_subject: str
    reply_body: str
    confidence_score: float
    needs_human_approval: bool
    proposal: Optional[ProposalOut] = None

# -----
# 2. MOTOR RAG E BANCO DE CONHECIMENTO
# -----

class VectorStorePB:
    def __init__(self):
        # Base de dados oficial inicial sobre a Educação em Paraíba
        self.kb: List[DocumentChunk] = [
            DocumentChunk(
                chunk_id="doc_pb_01",
                content="O Programa Alfabetiza Mais Paraíba busca garantir a alfabetização de 100% das crianças até o final do 2º ano do Ensino Fundamental nas redes municipais e estaduais.",
                metadata={"fonte": "SEECT-PB", "secao": "Alfabetização"}
            ),
            DocumentChunk(
                chunk_id="doc_pb_02",
                content="As Escolas Cidadãs Integrais (ECI) da Paraíba oferecem jornada escolar prolongada com 3 refeições diárias, laboratórios de ciências e orientação de projeto de vida.",
                metadata={"fonte": "SEECT-PB", "secao": "Ensino Integral"}
            ),
            DocumentChunk(
                chunk_id="doc_pb_03",
                content="O Calendário Escolar Oficial de Paraíba prevê 200 dias letivos obrigatórios, com início das aulas no mês de fevereiro e encerramento em dezembro.",
                metadata={"fonte": "SEECT-PB", "secao": "Calendário"}
            )
        ]

    def search(self, query: str) -> List[DocumentChunk]:
        # Algoritmo de busca por similaridade simplificado
        words = set(query.lower().split())
        results = []

```

```

        for doc in self.kb:
            doc_words = set(doc.content.lower().split())
            overlap = len(words.intersection(doc_words))
            if overlap > 0:
                results.append((doc, overlap))
        results.sort(key=lambda x: x[1], reverse=True)
        return [doc for doc, score in results]

    def add_chunk(self, content: str, source_info: str):
        new_id = f"doc_pb_{len(self.kb) + 1:02d}"
        self.kb.append(DocumentChunk(
            chunk_id=new_id,
            content=content,
            metadata={"fonte": source_info, "status": "Aprovado_Humano"}
        ))
        return new_id

# Instâncias Globais do Sistema
vector_store = VectorStorePB()
pending_proposals_list: List[ProposalOut] = []

# -----
# 3. FUNÇÕES AUXILIARES DE SANITIZAÇÃO E CÁLCULO
# -----

def sanitize_email_body(raw_text: str) -> str:
    """Remove assinaturas, cabeçalhos de resposta e caracteres indesejados."""
    # Remove citações de respostas anteriores
    cleaned = re.sub(r'Em \d{2}/\d{2}/\d{4}.*escreveu:', '', raw_text, flags=re.DOTALL)
    cleaned = re.sub(r'On.*wrote:', '', cleaned, flags=re.DOTALL)
    # Remove múltiplos espaços e quebras de linha excessivas
    cleaned = re.sub(r'\s+', ' ', cleaned).strip()
    return cleaned

def calculate_confidence(query: str, retrieved_docs: List[DocumentChunk]) -> float:
    """Calcula pontuação de confiança entre 0.0 e 1.0"""
    if not retrieved_docs:
        return 0.15
    query_words = set(query.lower().split())
    matches = set()
    for doc in retrieved_docs:
        matches.update(set(doc.content.lower().split()).intersection(query_words))

    match_ratio = len(matches) / max(len(query_words), 1)
    score = 0.40 + (0.55 * match_ratio)
    return round(min(score, 0.98), 2)

# -----
# 4. ENDPOINTS DA API REST
# -----

@app.post("/api/v1/process-email", response_model=AgentResponse)
def process_incoming_email(email_data: EmailIncoming, background_tasks: BackgroundTasks):
    """
    Endpoint chamado quando um novo e-mail chega ao servidor do Observatório.
    """
    cleaned_query = sanitize_email_body(email_data.body_text)
    relevant_docs = vector_store.search(cleaned_query)
    confidence = calculate_confidence(cleaned_query, relevant_docs)

    THRESHOLD = 0.75

    if confidence >= THRESHOLD:
        # Caso de alta confiança: Resposta direta
        context = "\n".join([d.content for d in relevant_docs])
        reply = (
            f"Prezado(a) responsável,\n\n"
            f"Em resposta à sua consulta enviada ao Observatório da Educação de  

            Paraiba:\n\n"
            f"{context}\n\n"
            f"Para mais informações detalhadas, acesse o nosso portal oficial.\n\n"
            f"Atenciosamente,\n"
            f"Equipe de Atendimento do Observatório da Educação de Paraiba"
        )
    return AgentResponse(
        recipient_email=email_data.sender_email,
        reply_subject=f"Re: {email_data.subject}",
    )

```

```

        reply_body=reply,
        confidence_score=confidence,
        needs_human_approval=False
    )

else:
    # Caso de baixa confiança: Envia acolhimento e cria proposta para revisão humana
    reply_contingency = (
        f"Prezado(a) responsável,\n\n"
        f"Recebemos sua mensagem referente a: '{cleaned_query}'.\n"
        f"Esta dúvida específica está sendo analisada pela equipe técnica da Secretaria
de Educação para atualização do nosso sistema.\n\n"
        f"Em breve enviaremos um retorno conclusivo.\n\n"
        f"Atenciosamente,\n"
        f"Equipe de Atendimento do Observatório da Educação de Paraíba"
    )

    proposal_id = len(pending_proposals_list) + 1
    new_proposal = ProposalOut(
        proposal_id=proposal_id,
        original_query=cleaned_query,
        suggested_text=f"Aclaração sobre '{cleaned_query}': [Inserir diretriz oficial
homologada pela SEECT-PB].",
        confidence_score=confidence,
        status="PENDENTE"
    )
    pending_proposals_list.append(new_proposal)

    return AgentResponse(
        recipient_email=email_data.sender_email,
        reply_subject=f"Re: {email_data.subject}",
        reply_body=reply_contingency,
        confidence_score=confidence,
        needs_human_approval=True,
        proposal=new_proposal
    )

@app.get("/api/v1/proposals", response_model=List[ProposalOut])
def list_pending_proposals():
    """Lista todas as propostas de atualização do texto-base pendentes de revisão
humana."""
    return [p for p in pending_proposals_list if p.status == "PENDENTE"]

@app.post("/api/v1/proposals/{proposal_id}/approve")
def approve_proposal(proposal_id: int):
    """Endpoint para o especialista humano aprovar uma proposta e enriquecer o texto-
base."""
    for prop in pending_proposals_list:
        if prop.proposal_id == proposal_id and prop.status == "PENDENTE":
            prop.status = "APROVADO"
            vector_store.add_chunk(
                content=prop.suggested_text,
                source_info=f"Validado_Humano_Query_{proposal_id}"
            )
    return {"message": f"Proposta #{proposal_id} aprovada! Texto-base do
Observatório atualizado com sucesso."}

    raise HTTPException(status_code=404, detail="Proposta não encontrada ou já
processada.")

# -----
# 5. EXECUÇÃO DA APLICAÇÃO
# -----
if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main_api:app", host="0.0.0.0", port=8000, reload=True)

```

## 5. Governança de Dados, Privacidade (LGPD) e Mitigação de Riscos

Ao lidar com informações provenientes do ambiente educacional e interações com pais e responsáveis, o projeto observa estritamente a **Lei Geral de Proteção de Dados Pessoais (LGPD - Lei nº 13.709/2018)** do Brasil.

### 5.1. Proteção à Privacidade e Dados de Menores

O Estatuto da Criança e do Adolescente (ECA) e a LGPD estabelecem proteção reforçada aos dados pessoais de crianças e adolescentes. O agente inteligente implementa os seguintes controles:

- **Anonimização na Ingestão:** Todos os nomes de alunos, números de matrícula (CPF/RA) e dados sensíveis de saúde contidos nos e-mails são substituídos por tokens anônimos (ex.: [ALUNO\_ANON\_01]) antes da análise pelo modelo de IA.
- **Princípio da Minimização:** O sistema retém apenas o conteúdo da dúvida técnica e exclui identificadores pessoais das bases de enriquecimento de conhecimento.
- 

### 5.2. Prevenção contra *Prompt Injection* e Envenenamento de Dados

A arquitetura desacopla totalmente o pipeline de recebimento de mensagens da base vetorial de produção. O texto sugerido pelo agente entra obrigatoriamente em um estado de quarentena. Nenhuma instrução contida no e-mail do usuário pode alterar diretamente o comportamento das regras do sistema (*System Prompts*) nem ser gravada na base de dados de produção sem o aval do revisor humano credenciado.

## 6. Referências Bibliográficas (Norma APA 7ª Edição)

- Brasil. (2018). *Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)*. Diário Oficial da União.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Lewis, P., Perez, E., Pineda, A., Petroni, F., Riedel, S., Goyal, N., Fan, A., Lewis, M., Yih, W. T., Rocktäschel, T., Karpukhin, V., & Riedel, S. (2020). Retrieval-

augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

- Mendoza, C., & Santos, R. (2023). Governança e inteligência artificial no setor público brasileiro: Transparência e eficiência na gestão educacional. *Revista Brasileira de Administração Pública*, 57(3), 412–435.
- Paraíba. Secretaria de Estado da Educação e da Ciência e Tecnologia (SEECT-PB). (2024). *Observatório da Educação de Paraíba: Diretrizes e Indicadores Pedagógicos*. Governo do Estado de Paraíba.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
- Wu, X., & Zhang, Y. (2022). Human-in-the-loop artificial intelligence: A survey of taxonomy, architectures, and applications. *ACM Computing Surveys*, 55(8), 1–37.